34. Design LinkedIn News Feed

Key Terms:

Term	Full Name	What it does
Fanout	Content distribution	Pushing posts to followers' feeds
Fanout-on-Write	Push model	Pre-compute feeds when content is posted
Fanout-on-Read	Pull model	Build feed when user requests it
Celebrity Problem	High-follower users	Users with millions of followers (expensive fanout)
Feed Ranking	Personalization	ML model ordering content by relevance
Time Decay	Freshness signal	Older posts score lower in ranking

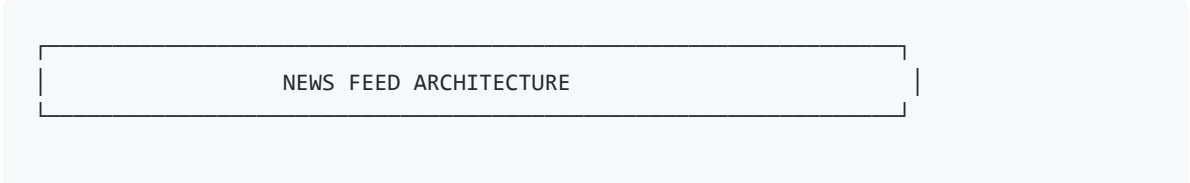
Problem Statement

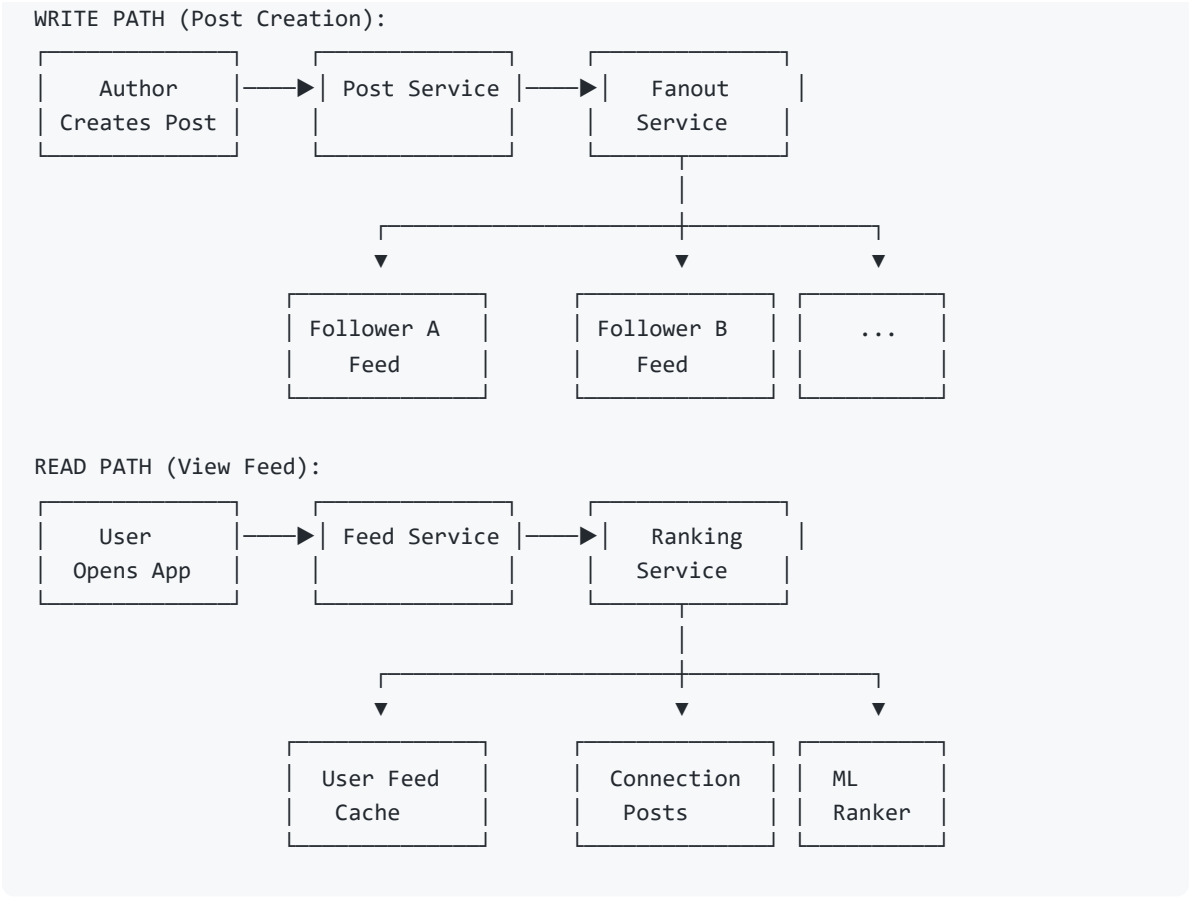
Design a personalized news feed for LinkedIn that shows relevant posts, articles, and updates from connections and followed entities.

Clarifying Questions to Ask

1. **Scale**: How many users? Posts per day? (800M users, 2M posts/day?)
2. **Content types**: Posts, articles, job updates, ads?
3. **Personalization**: How personalized should it be?
4. **Freshness**: Real-time or eventual consistency OK?
5. **Engagement**: Prioritize engagement or chronological?
6. **Connections**: Average connections per user? (500?)
7. **Read/write ratio**: Mostly reading feeds?

High-Level Design





API Design

Feed APIs

Method	Endpoint	Description
`GET`	`/api/v1/feed?cursor={cursor}&limit=20`	Get personalized feed
`GET`	`/api/v1/feed/refresh`	Get new posts since last fetch

Post APIs

Method	Endpoint	Description
`POST`	`/api/v1/posts`	Create a post
`GET`	`/api/v1/posts/{postId}`	Get post details
`PUT`	`/api/v1/posts/{postId}`	Edit a post
`DELETE`	`/api/v1/posts/{postId}`	Delete a post

Engagement APIs

Method	Endpoint	Description
`POST`	`/api/v1/posts/{postId}/like`	Like a post
`DELETE`	`/api/v1/posts/{postId}/like`	Unlike a post
`POST`	`/api/v1/posts/{postId}/comments`	Comment on a post
`POST`	`/api/v1/posts/{postId}/share`	Share/repost
`GET`	`/api/v1/posts/{postId}/comments?cursor={c}`	Get comments

Create Post Request

```
POST /api/v1/posts
{
  "content": "Excited to share...",
  "media": [{ "type": "image", "url": "..." }],
  "visibility": "connections",  // public, connections, private
  "mentions": ["user_456"],
  "hashtags": ["systemdesign"]
}
```

Key Components

Component	Purpose	Technology
Post Service	Create/store posts	PostgreSQL
Fanout Service	Distribute to feeds	Kafka + Workers
Feed Cache	Pre-computed feeds	Redis
Graph Service	Connections, follows	Graph DB
Ranking Service	Personalized ordering	ML model
Search Service	Content discovery	Elasticsearch

Fanout Strategies

```
FANOUT ON WRITE (Push model):
┌─────────────────────────────────────────────────────────────┐
│  When user posts:                                           │
│    For each follower:                                      │
│      Add post_id to follower's feed cache                  │
```

```
│                                                                  │
│  Pros: Fast reads (feed is pre-built)                            │
│  Cons: Expensive for users with many followers (celebrities)     │
│                                                                  │
│  Best for: Users with < 10K followers                            │
└──────────────────────────────────────────────────────────────────┘

FANOUT ON READ (Pull model):
┌──────────────────────────────────────────────────────────────────┐
│  When user opens feed:                                           │
│    Fetch posts from all followed users                           │
│    Merge and rank                                                │
│                                                                  │
│  Pros: No fanout cost for celebrities                            │
│  Cons: Slower reads, more compute per request                    │
│                                                                  │
│  Best for: Following celebrities (Bill Gates, etc.)              │
└──────────────────────────────────────────────────────────────────┘

HYBRID (LinkedIn approach):
┌──────────────────────────────────────────────────────────────────┐
│  - Regular users: Fanout on write                                │
│  - Celebrities (>10K followers): Fanout on read                  │
│  - Merge at read time                                            │
│                                                                  │
│  User opens feed:                                                │
│    1. Fetch pre-built feed (from regular connections)            │
│    2. Fetch recent posts from followed celebrities               │
│    3. Merge and rank                                             │
└──────────────────────────────────────────────────────────────────┘
```

Feed Ranking

```
RANKING SIGNALS:
┌──────────────────────────────────────────────────────────────────┐
│  Relevance Score = Σ (weight_i * signal_i)                       │
│                                                                  │
│  Signals:                                                        │
│  ┌──────────────────────────────────────────────────────────────┐│
│  │ Relationship strength:                                       ││
│  │   - Connection degree (1st, 2nd, 3rd)                        ││
│  │   - Interaction history (messages, profile views)            ││
│  │   - Common connections                                       ││
│  │                                                              ││
│  │ Content quality:                                             ││
│  │   - Engagement rate (likes, comments, shares)                ││
│  │   - Content type affinity (user prefers articles vs posts)   ││
│  │   - Topic relevance (matches user interests)                 ││
│  │                                                              ││
│  │ Freshness:                                                   ││
│  │   - Time decay (recent posts score higher)                   ││
│  │   - Trending boost (viral content)                           ││
│  │                                                              ││
```

```
│   │ Diversity:                                                    ||
│   │   - Don't show 5 posts from same author in a row            ||
│   │   - Mix content types (posts, articles, jobs)               ||
│   └──────────────────────────────────────────────────────────────┘|
└──────────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Feed cache**: Store top 500 post IDs per user in Redis
- **Sharding**: Shard by user_id for feed storage
- **Async fanout**: Queue-based fanout with rate limiting
- **Read replicas**: Separate read/write paths
- **Edge caching**: Cache popular content at CDN

Trade-offs

Decision	Option A	Option B	Recommendation
Fanout	On write	On read	**Hybrid** - best of both
Ranking	Chronological	ML-ranked	**ML-ranked** - better engagement
Cache	Full posts	Post IDs only	**Post IDs** - less memory
Freshness	Real-time	Near-real-time	**Near-real-time** - simpler

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Pre-computed feed cache storage	Tech Guide
Kafka	Async fanout and event distribution	Tech Guide
Cassandra	Post and feed persistence	Tech Guide

How to Explain It Well

> **Opening**: "LinkedIn feed needs fast reads with personalized ranking. I'll use hybrid fanout and ML-based ranking."

> **Key talking points**:
>
> - "Hybrid fanout: push for regular users, pull for celebrities"
> - "ML ranking combines relationship strength, content quality, freshness"
> - "Feed cache stores top 500 post IDs, hydrate on read"
> - "Async fanout via Kafka to handle post burst"
> - "Diversity rules prevent feed dominated by single author"
